AgileFlow AI

Human-in-the-Loop Multi-Agent Framework for Agile SDLC Automation

System Architecture Diagram and Component Relationship Summary

File 03	Knowledge-passing document for architecture, component relationships, approval gates, orchestrator, shared knowledge base and feedback loops.

1. Purpose of This Document

This document summarizes the high-level architecture of AgileFlow AI and explains how the major components are connected. It is designed for knowledge passing before UI design, backend implementation, API planning, and agent-level development.

The focus is the whole-system structure, not the detailed internal sub-objectives of each agent. Use this file together with the UI/Figma plan, implementation working plan, API contract, and data schema plan.

2. Architecture Summary

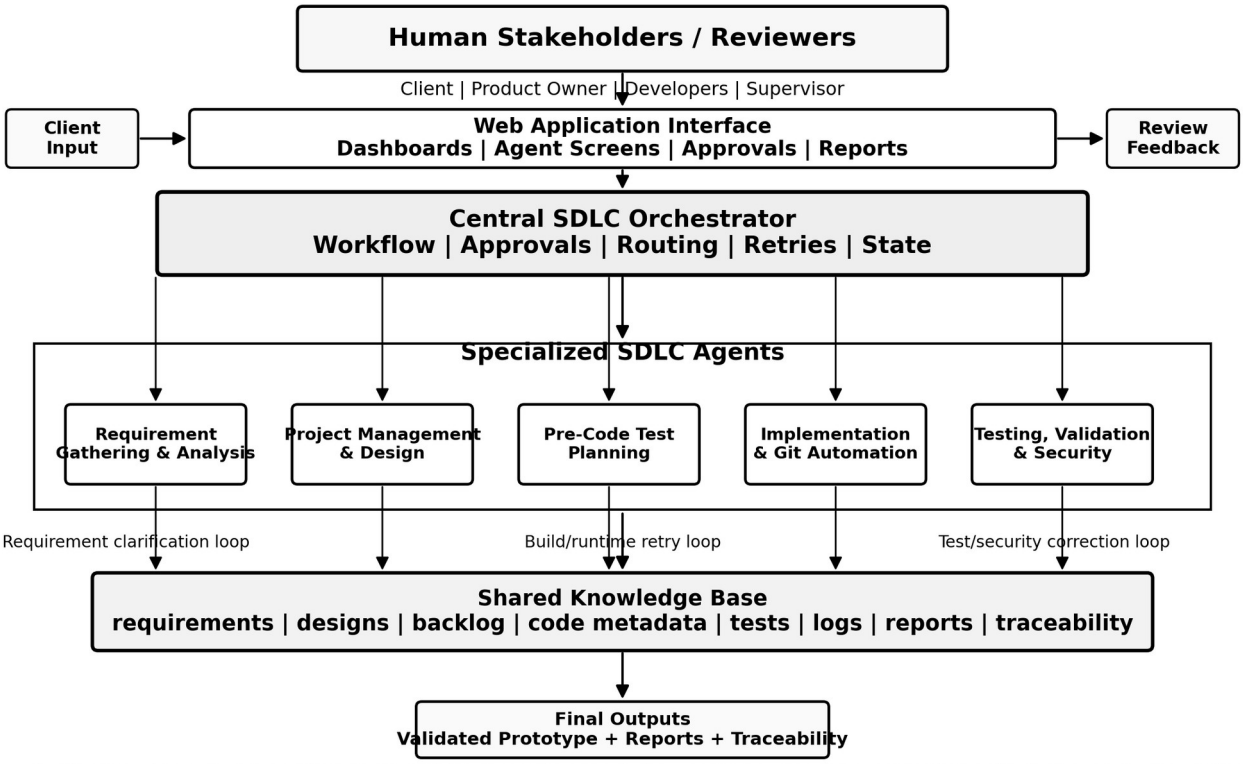
AgileFlow AI is a web-based, human-in-the-loop, multi-agent SDLC automation platform. The system receives raw client input, transforms it through a controlled Agile workflow, and produces a validated runnable prototype foundation with reports and traceability evidence.

- Human stakeholders review and approve critical outputs before the workflow continues.
- The Central SDLC Orchestrator controls workflow order, handovers, approvals, failures, retries, and state.
- Five major agent stages transform the project from client idea to validated prototype.
- The Shared Knowledge Base stores approved artifacts, logs, reports, and traceability links.
- Feedback loops route requirement problems, build failures, test failures, and security findings to the correct component.

3. High-Level System Architecture Diagram

This diagram shows the main architecture layers: human stakeholders, central orchestrator, specialized agents, shared knowledge base, and final outputs.

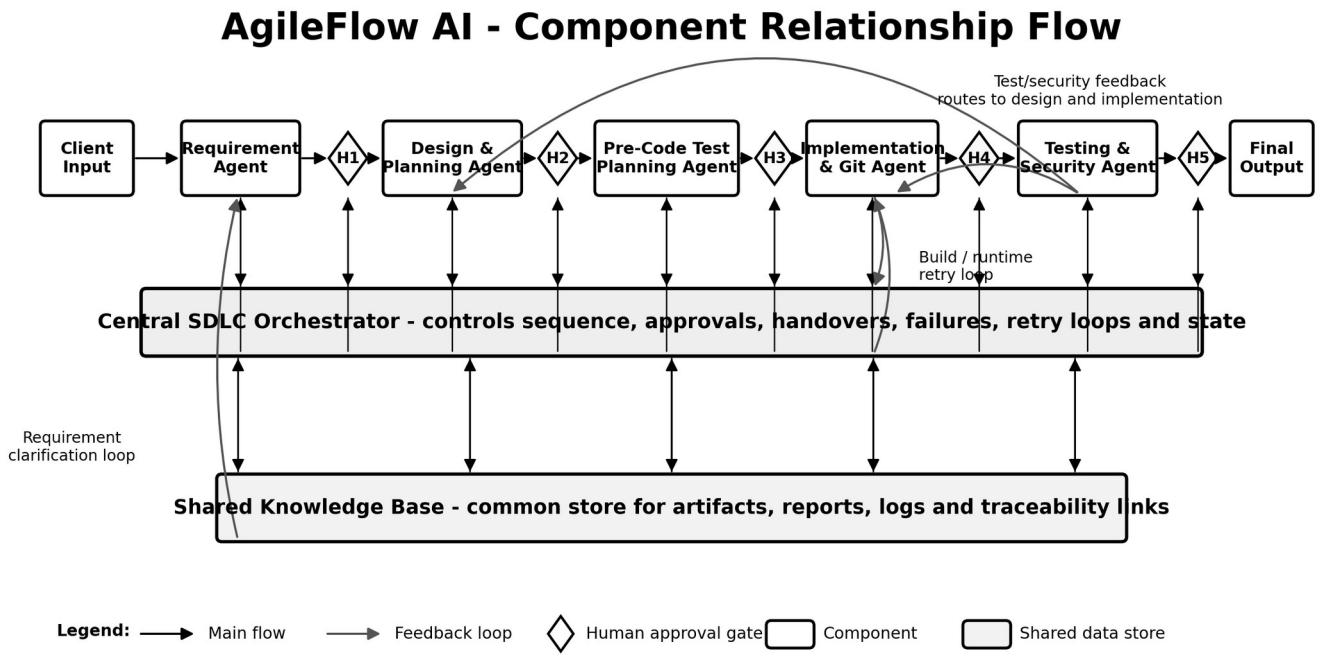
AgileFlow AI - High-Level System Architecture



Architecture view: users interact through the web interface; the orchestrator coordinates all agents; agents share artifacts through the knowledge base.

4. Component Relationship Flow Diagram

This diagram shows how the main components are linked in the end-to-end workflow. The Testing, Validation and Security Agent appears in two logical stages: pre-code test planning before implementation and post-code validation after implementation.



5. Main Components and Responsibilities

Component	System-Level Responsibility
Client Input	Starting point. Accepts text, documents, web links, audio, video, and live interview responses.
Requirement Gathering and Analysis Agent	Transforms client input into structured requirements, user stories, acceptance criteria, requirement IDs, and quality findings.
Project Management and Design Agent	Converts approved requirements into backlog items, sprint plans, design artifacts, API plans, database schemas, UI flows, and traceability mappings.
Pre-Code Test Planning Agent	Prepares test specifications, security test cases, test skeletons, and requirement-to-test mappings before implementation starts.
Implementation and Git Automation Agent	Generates selected prototype modules, manages repository structure, branches, commits, build/runtime logs, and limited self-fix loops.
Testing, Validation and Security Agent	Executes tests, analyzes failures, generates bug reports, performs security validation, and produces final validation evidence.
Central SDLC Orchestrator	Controls workflow transitions, approval gates, handovers, failure routing, retry loops, and project state.
Shared Knowledge Base	Stores requirements, design artifacts, Jira items, test cases, code metadata, logs, bug reports, security findings, and traceability links.
Final Output	Provides structured Agile project environment, validated prototype foundation, reports, and traceability evidence.

6. Human Approval Gates

Human approval gates keep the system controlled and reviewable. The system should not automatically accept major generated artifacts without review.

Gate	Approval Point	Purpose
H1	Requirement Approval	After requirements, user stories, acceptance criteria, and quality issues are generated.
H2	Design and Planning Approval	After backlog, sprint plan, design artifacts, API/database/UI outputs, and traceability are generated.
H3	Pre-Code Test Approval	After test specifications, test skeletons, security tests, and requirement-to-test mapping are generated.
H4	Implementation Approval	After selected backend/frontend prototype modules, Git metadata, and build/runtime logs are produced.
H5	Final Validation Approval	After testing, bug reporting, security validation, and final evidence generation are completed.

7. Shared Knowledge Base Content

The Shared Knowledge Base acts as the common artifact store. It prevents isolated agent outputs and enables traceability across the SDLC.

- Requirements: REQ IDs, functional requirements, non-functional requirements, user stories, acceptance criteria, ambiguity and missing-information reports.
- Designs: UML diagrams, API specifications, database schemas, UI flows, wireframe references, and design decisions.
- Planning: Jira-style epics, stories, tasks, subtasks, bugs, sprint plan, and priorities.
- Implementation: generated modules, file paths, commit metadata, branch history, build logs, and runtime logs.
- Testing and Security: test cases, test results, coverage evidence, bug reports, security findings, vulnerability reports, and regression results.
- Traceability: requirement-to-design-to-code-to-test-to-bug/security links.

8. Feedback and Correction Relationships

Issue Type	Detected By	Routed To	Correction Action
Requirement problem	Human / Orchestrator	Requirement Gathering and Analysis Agent	Clarify, refine, regenerate, and re-approve requirements.
Design or planning mismatch	Human / Orchestrator / test feedback	Project Management and Design Agent	Update backlog, design artifact, API/DB/UI plan, or Jira issue.
Build or runtime failure	Implementation Agent logs	Implementation and Git Automation Agent	Analyze logs, apply limited self-fix, commit fix, and rerun.
Failed tests	Testing Agent	Orchestrator -> Design Agent + Implementation Agent	Create/update bug item, fix code, rerun tests and regression tests.
Security finding	Testing and Security Agent	Orchestrator -> Design Agent + Implementation Agent	Track security issue, fix code/configuration, rerun security checks.

9. Implementation Notes for the Development Team

- Implement the Orchestrator and Shared Knowledge Base early because all agents depend on them.
- Use fixed workflow stages and approval gate names: H1, H2, H3, H4, and H5.
- Keep the Testing, Validation and Security Agent as one research component, even though it appears before and after implementation.
- Do not allow agents to store outputs only in local UI state; every approved artifact must be saved to the Shared Knowledge Base.
- Every generated artifact must contain traceability IDs where possible: REQ, US, AC, DES, API, DB, TASK, TEST, CODE, BUG, SEC, TRACE.
- For the first MVP, use mock outputs where needed, but preserve the full workflow and data relationships.

10. Scope Boundary

AgileFlow AI should be presented as an Agile SDLC automation framework that creates a structured project environment and validated prototype foundation. It should not be presented as a fully autonomous replacement for a complete software development team or as a final production-ready software builder.

Developers, clients, supervisors, and reviewers remain responsible for approving requirements, designs, tests, implementation outputs, and final validation reports.